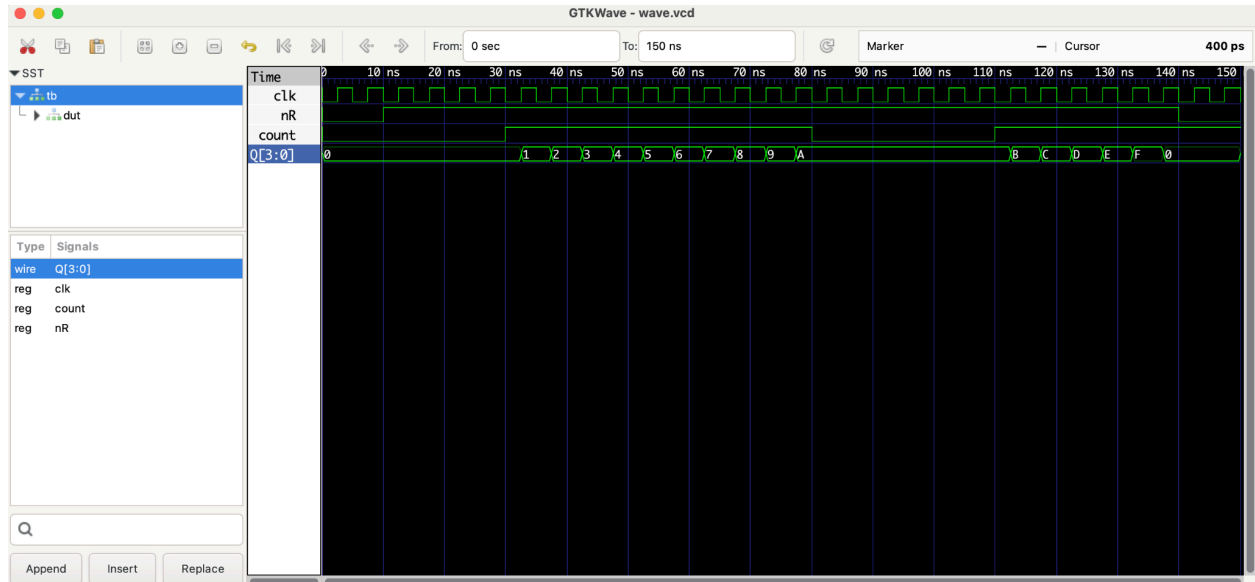
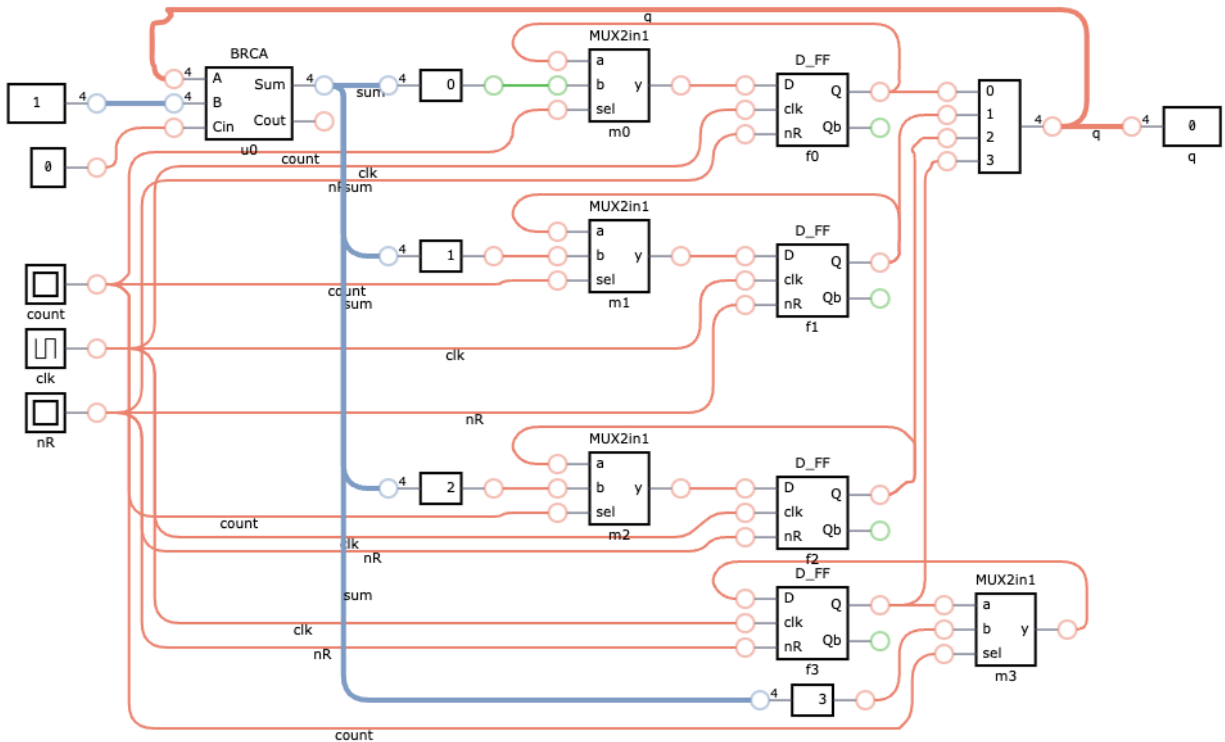


CMPE 124 LAB 4: 4-Bit Counter

Waveform Test for Specified Design Criteria:



Schematic (Generated via DigitalJS Online):



Truth Table:

t	clk	nR	count	Q
3000	1	0	0	0000
8000	1	0	0	0000
13000	1	1	0	0000
18000	1	1	0	0000
23000	1	1	0	0000
28000	1	1	0	0000
33000	1	1	1	0000
38000	1	1	1	0001
43000	1	1	1	0010
48000	1	1	1	0011
53000	1	1	1	0100
58000	1	1	1	0101
63000	1	1	1	0110
68000	1	1	1	0111
73000	1	1	1	1000
78000	1	1	1	1001
83000	1	1	0	1010
88000	1	1	0	1010
93000	1	1	0	1010
98000	1	1	0	1010
103000	1	1	0	1010
108000	1	1	0	1010
113000	1	1	1	1010
118000	1	1	1	1011
123000	1	1	1	1100
128000	1	1	1	1101
133000	1	1	1	1110
138000	1	1	1	1111
140000	0	0	1	0000
143000	1	0	1	0000
148000	1	0	1	0000

Logic:

- 0 is the controlling value, so low active: when $R=0 \rightarrow$ Cout ALWAYS =0, regardless of Count
 - When $R=1$, Count=1, Cout should increment from 0000 to 1111 one digit at a time, then back to 0000
 - When $R=1$, Count=0, Cout should stall
- We have one 4-bit RCA followed by four 2-1 MUXes, each connected to a D Flip-Flop.

Explanation:

In the previous lab, we made a ripple carry adder using primitive gates (asynchronous circuit), where the carry “ripples” from one full adder to the next, resulting in significant delay since the carry has to propagate through every stage. In this lab, we are building a 4-bit *counter* instead of an adder, which is built from flip-flops and has the ability to store the current count. Whereas the final output for the ripple carry adder is *not stable* until the carry has propagated through all stages since the sum and carry take time to ripple through the chain of adders, the synchronous 4-bit counter has all flip-flops clocked at the same time by a single, common clock signal. This makes synchronous counters faster and more stable because they can change states simultaneously based on a clock.

Lab 4 requires us to build a 4-Bit Counter using RTL (Register-Transfer Level) Verilog, which focuses on what the circuit does rather than how it's implemented at the gate level by describing the circuit in terms of data flow between registers and combinational logic. Our 4-Bit Counter consists of one 4-bit RCA, four 2-1 MUXes, and four D-type flip-flops in succession, with Reset, Clock, and Count inputs for the counters.

Design Code:

```
`timescale 1ns/1ps
`default_nettype wire

//A + B + Cin -> {Cout, Sum}

module BRCA(
    //wire connects signals, cannot assign in always blocks
    //no storage in wire
    input wire [3:0] A,
    input wire [3:0] B,
    input wire Cin,
    output wire [3:0] Sum,
    output wire Cout
);

    //sum and Cout are changeable in always loops
    //reg = flip flop or latch, holds values until new assign
    //reg assigned in procedural blocks
    reg [3:0] sum_r;
    reg cout_r;

    integer i;
    //initial carry in assigned to intermediate carry variable c
    reg c; //running carry

    //identifies all signals read on RHS of assignments in always block
```

```

always @(*) begin
    //triggered when Cin
    c = Cin;
    //4 full adder stages
    for(i=0; i<4; i++)begin
        //binary addition '+'
        //concatenation assigns 2-bit result: higher = c, lower = sum_r[i]
        {c, sum_r[i]} = A[i] + B[i] + c;
    end
    cout_r = c;
end

assign Sum = sum_r;
assign Cout = cout_r;

endmodule

```

```

`timescale 1ps/1ps
`default_nettype wire

//1-it reg w load enable
//sel = 1 at posedge clk -> load D
//sel -0 at posedge clk -> hold prev Q

module MUX2in1(
input a,b, sel,
output reg y
);
    always @(*) begin
        if(sel)
            y=b;
        else
            y=a;
        end
    endmodule

```

```

`timescale 1ps/1ps

```

```

`default_nettype wire

//Q=w4, Qb=W3
module D_FF(input D, clk, nR, output reg Q, Qb);
assign Qb = ~Q;
//rising edge triggered register (transparent/ dont care when clk=1)
always @ (posedge clk or negedge nR) begin
    if(!nR)
        Q <= 1'b0;
    else
        Q <=D;
end

endmodule

```

```

`timescale 1ns/1ps

module COUNTER(
    input wire clk,
    input wire nR,
    input wire count,
    output wire [3:0] q
);
    wire [3:0] sum;
    wire [3:0] nxt;
    wire cout;

    BRCA u0(.A(q), .B(4'b0001), .Cin(1'b0), .Sum(sum), .Cout(cout));

    // want: count=1 → use sum; count=0 → use q
    MUX2in1 m0(.a(q[0]), .b(sum[0]), .sel(count), .y(nxt[0]));
    MUX2in1 m1(.a(q[1]), .b(sum[1]), .sel(count), .y(nxt[1]));
    MUX2in1 m2(.a(q[2]), .b(sum[2]), .sel(count), .y(nxt[2]));
    MUX2in1 m3(.a(q[3]), .b(sum[3]), .sel(count), .y(nxt[3]));

    D_FF f0(.D(nxt[0]), .clk(clk), .nR(nR), .Q(q[0]), .Qb());
    D_FF f1(.D(nxt[1]), .clk(clk), .nR(nR), .Q(q[1]), .Qb());
    D_FF f2(.D(nxt[2]), .clk(clk), .nR(nR), .Q(q[2]), .Qb());
    D_FF f3(.D(nxt[3]), .clk(clk), .nR(nR), .Q(q[3]), .Qb());

endmodule

```

Testbench Code:

```
`timescale 1ns/1ps

module tb;

    reg clk;
    reg count;
    reg nR;          // active-low
    wire [3:0] Q;

    COUNTER dut(.clk(clk), .nR(nR), .count(count), .q(Q));

    initial begin
        clk = 0;
        forever #2.5 clk = ~clk;
    end

    initial begin
        $dumpfile("wave.vcd");
        $dumpvars(0, tb);

        // Hold reset low initially → Q = 0000
        nR = 0; count = 0;
        #10 nR = 1; count = 0; // release reset, still stall
        #20 nR = 1; count = 1; // start counting
        #50 nR = 1; count = 0; // stall, Q should hold
        #30 nR = 1; count = 1; // resume counting
        #30 nR = 0; count = 1; // assert reset again → Q goes to 0000
        #10 $finish;
    end

    // print a line on every clock edge (and when reset drops)
    always @(posedge clk or negedge nR) begin
        $display("%4t  %b  %b  %b  %b",
                $time, clk, nR, count, Q);
    end

endmodule
```